Practical Machine Learning Project

Irene Mommers

2026-01-13

Data was downloaded and stored into the working directory. I will use the 'pml-training.csv' to partition in a training and test dataset, where the trainingset will be used for development of seperate models, and the trainingset to fit those models and combine them into one predictor. The other dataset ('pml-testing.csv') will be used as validationset and thus not be touched before application of the final model in the end.

Read and explore dat

```
# load data
dat = read.csv("pml-training.csv")

# partition
inBuild <- createDataPartition(y=dat$classe, p=0.75, list=FALSE)
train <- dat[inBuild,]
test  <- dat[-inBuild,]

# set seed for reproducibility
set.seed(75129)

# inspect size of train and test data
dim(train); dim(test)
```

```
## [1] 14718   160
```

```
## [1] 4904  160
```

At first glance, 19622 observation and 160 variables means that there are plenty of observations per variable, yet it might still be good to cut down the number of variables to include in the model, to prevent overfitting.

Looking at the data (View(train)), it is noticable that there appears to be loads of missingness in some variables and that some variables show little variation. Also, there are time-stamp variables and a character with the date and time - this last one can be removed as it is non-informative as compared to the others.

```
# set outcome variable to factor
train$classe <- as.factor(train$classe)

# Remove id and timestamps/windows as these will probably not be predictive
train <- train %>% select(-X,-user_name,-raw_timestamp_part_1,
-raw_timestamp_part_2,-cvtd_timestamp,-new_window,-num_window)

# explore missingness
table(colSums(is.na(train)))
```

```
##
##     0 14413
##    86    67
```

```
# drop variables with >95% missing
train <- train[,!((colSums(is.na(train))/nrow(train))>0.95)]
dim(train)
```

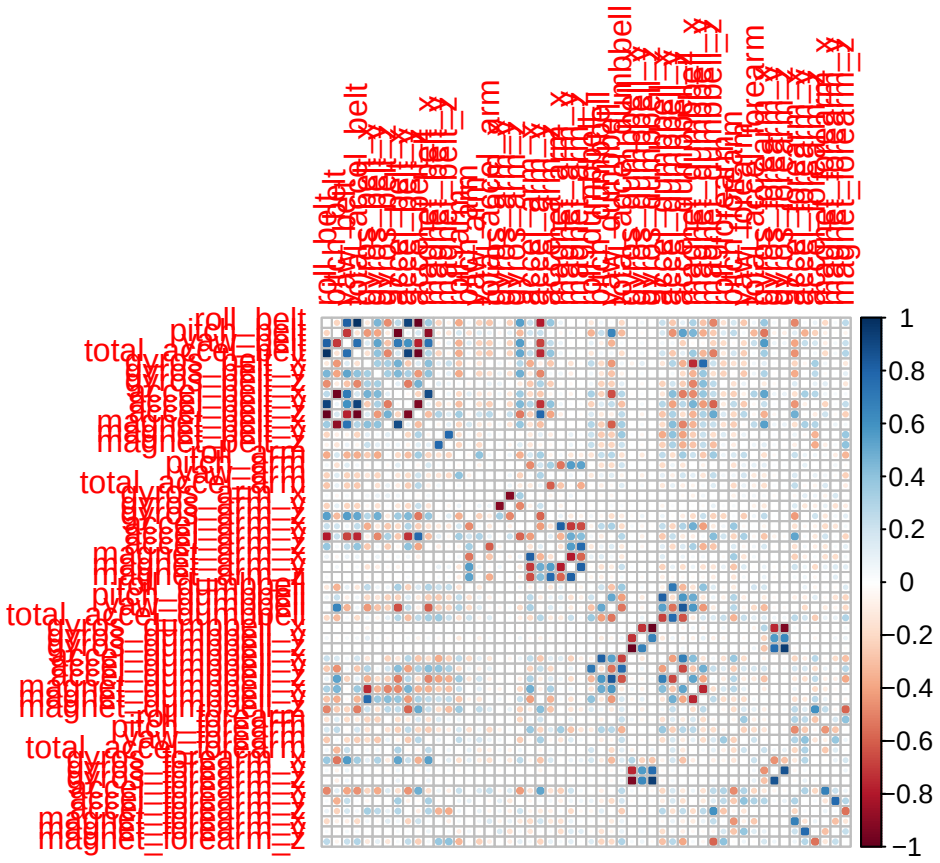
```
## [1] 14718    86
```

```
# explore near-zero variance
nsv <- nearZeroVar(train, saveMetrics=TRUE)

# drop variables from train with
train <- train[,!(unlist(nsv$nzv))]
dim(train)
```

```
## [1] 14718    53
```

```
# explore correlatedness of numeric variables
num <- train[,sapply(train,is.numeric)] # obtain numeric columns
num <- as.data.frame(sapply(num, scale)) # scale variables
cor.table <- cor(num)
diag(cor.table) <-  0 # set the diagonal to 0
corrplot(cor.table)
```



```
which(cor.table > 0.8, arr.ind=T)
```

```
##                   row col
## yaw_belt            3   1
## total_accel_belt    4   1
## accel_belt_y        9   1
## roll_belt           1   3
## roll_belt           1   4
## accel_belt_y        9   4
## magnet_belt_x      11   8
## roll_belt           1   9
## total_accel_belt    4   9
## accel_belt_x        8  11
## magnet_arm_x       24  21
## accel_arm_x        21  24
## magnet_arm_z       26  25
## magnet_arm_y       25  26
## accel_dumbbell_x   34  28
## accel_dumbbell_z   36  29
## gyros_forearm_z    46  33
## pitch_dumbbell     28  34
## yaw_dumbbell       29  36
## gyros_forearm_z    46  45
## gyros_dumbbell_z   33  46
## gyros_forearm_y    45  46
```

Cross validation & model fitting

cross validation needed to be applied. 10-fold cross validation is quite common, thus let's go for 10 folds, using the trainControl() function from the caret package.

As there were only variables where >19 000 cases were missing and thus were dropped or without missings - thus there's no need for imputation.

Combining predictors proved worthwhile during the course, so I will combine two models: a random forest and a linear discriminant analysis.

Tree-based models generally do not benefit much from principle component analysis, while linear models are sensitive to highly correlated variables . Since there are quite a lot of highly correlated variables and the goal is to make good predictions (not necessarily to build an interpretable model), I have added principle component analysis to the non-tree-based models. I do not want to lose much information, so I used a high threshold (0.95, keeping 95% of the variance).

To ensemble the models, in the course method='gam' was used. However, generalized additive models are a binary classifier. That is why here I used 'multinom' instead.

```
# cross validation
train_cv <- trainControl(
  method = "cv",
  number = 10
)

# models
# gradient boosting machine
```

```r
modFit_gbm <- train(classe ~., method=c("gbm"), trControl=train_cv, data=train, verbose=0)
# lda
modFit_lda <- train(classe ~., method=c("lda"), preprocess="PCA", thresh = 0.95,trControl=train_cv, data

# predict on test data
predtestgbm <- predict(modFit_gbm, test)
predtestlda <- predict(modFit_lda, test)

# data frame that combines the predictors
predDf <- data.frame(gbm=predtestgbm, lda=predtestlda,  classe=test$classe)

# cross validation
test_cv <- trainControl(
  method = "cv",
  number = 10
)
# Fit model that combines predictors
combFit <- train(classe~., method="multinom", trControl=test_cv, data=predDf, trace=FALSE)
# Make predictions
predtest <- predict(combFit, predDf)
# Obtain accuracy and other metrics
confusionMatrix(predtest, as.factor(test$classe))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    A    B    C    D    E
##          A 1379   30    0    1    1
##          B   12  890   29    4   10
##          C    4   27  812   15   16
##          D    0    1   13  771    6
##          E    0    1    1   13  868
##
## Overall Statistics
##
##                Accuracy : 0.9625
##                  95% CI : (0.9568, 0.9676)
##     No Information Rate : 0.2845
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9525
##
##  Mcnemar's Test P-Value : 2.636e-05
##
## Statistics by Class:
##
##                      Class: A Class: B Class: C Class: D Class: E
## Sensitivity            0.9885   0.9378   0.9497   0.9590   0.9634
## Specificity            0.9909   0.9861   0.9847   0.9951   0.9963
## Pos Pred Value         0.9773   0.9418   0.9291   0.9747   0.9830
## Neg Pred Value         0.9954   0.9851   0.9893   0.9920   0.9918
## Prevalence             0.2845   0.1935   0.1743   0.1639   0.1837
```

```
## Detection Rate         0.2812   0.1815   0.1656   0.1572   0.1770
## Detection Prevalence   0.2877   0.1927   0.1782   0.1613   0.1801
## Balanced Accuracy      0.9897   0.9620   0.9672   0.9770   0.9798
```

Out of sample error

Although on the testset the accuracy of the combined model is 0.96, it is likely that some overfitting will have taken place. However, with 25 variables and 19622 observations, it probably will still be a feasible model (>0.8 accuracy) when applied out of sample.

The code below shows predictions performed using the validation dataset.

```
# load data for testing
validation = read.csv("pml-testing.csv")

# predict on validation data
predvalgbm <- predict(modFit_gbm, validation)
predvallda <- predict(modFit_lda, validation)

# Combined prediction data
predvalDf <- data.frame(gbm=predvalgbm, lda=predvallda)

# predict
predval <- predict(combFit, newdata=predvalDf)
```